ДИСЦИПЛИНА **Конфигурационное управление**

(полное наименование дисциплины без сокращений)

ИНСТИТУТ **Информационных технологий**

КАФЕДРА **Корпоративных информационных систем**

(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Задание для текущего контроля**

(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **П.Н. Советов**

(фамилия, имя, отчество)

СЕМЕСТР **3 семестр (осенний) 2026/2027 учебного года**

(указать семестр обучения, учебный год)

Конфигурационное управление

Сборник практических работ № 1

ИКБО-42-25

РТУ МИРЭА – 2026

Оглавление

О практических работах 4
Перечень нарушений оформления git-репозитория 5
Вариант №1 8
Вариант №2 11
Вариант №3 14
Вариант №4 17
Вариант №5 20
Вариант №6 23
Вариант №7 26
Вариант №8 29
Вариант №9 32
Вариант №10 35
Вариант №11 38
Вариант №12 41
Вариант №13 44
Вариант №14 47
Вариант №15 50
Вариант №16 53
Вариант №17 56
Вариант №18 59
Вариант №19 62
Вариант №20 65
Вариант №21 68
Вариант №22 71
Вариант №23 74
Вариант №24 77
Вариант №25 80

Вариант №26 83
Вариант №27 86
Вариант №28 89
Вариант №29 92
Вариант №30 95
Вариант №31 98
Вариант №32 101
Вариант №33 104
Вариант №34 107
Вариант №35 110
Вариант №36 113
Вариант №37 116
Вариант №38 119
Вариант №39 122
Вариант №40 125

О практических работах

Практические работы (ПР) состоят из нескольких этапов. ПР выполняются очно и защита каждого этапа происходит на семинарских занятиях. Этапы работы над ПР сохраняются в публично доступном git-репозитории. Каждый этап разработки ПР должен быть отражен в истории коммитов с детальными сообщениями. Студент самостоятельно выбирает язык реализации.

Документация оформляется в виде readme.md, который содержит:

1. Общее описание.
2. Описание всех функций и настроек.
3. Описание команд для сборки проекта и запуска тестов.
4. Примеры использования.

Список публичных git-сервисов для репозиториев:

1. github.com
2. gitea.com
3. gitlab.com
4. gitflic.ru
5. hub.mos.ru
6. gitverse.ru
7. gitee.com

Репозиторий должен быть оформлен стандартно и не должен содержать типичных нарушений, перечисленных в разделе «Перечень нарушений оформления git-репозитория».

Перечень нарушений оформления git-репозитория

№	Описание правила
Структура репозитория	
С1.	В репозитории отсутствует файл README.md
С2.	В репозитории отсутствует файл .gitignore
С3.	В репозитории отсутствует папка с исходным кодом src
С4.	В репозитории отсутствует папка с тестами tests
С5.	В репозитории отсутствует скрипт запуска run.sh, run.bat или Makefile
С6.	Коммиты не соответствуют спецификации Conventional/Scoped Commits
Сгенерированные файлы	
Г1.	Репозиторий содержит архивы
Г2.	Репозиторий содержит бинарные файлы
Г3.	Репозиторий содержит служебные файлы языка Python
Г4.	Репозиторий содержит артефакты сборки
Г5.	Репозиторий содержит файлы с настройками редактора кода
Рецензирование	
Р1.	Репозиторий содержит слишком большие файлы (>1000 строк)
Р2.	Код на языке Python содержит слишком длинные строки (>80 символов)
Р3.	Код на языке JavaScript содержит слишком длинные строки (>80 символов)
Р4.	Код на языке Kotlin содержит слишком длинные строки (>100 символов)
Р5.	Код на языке Rust содержит слишком длинные строки (>100 символов)
Р6.	Код в репозитории содержит слишком длинные строки (>120 символов)
Функции	
Ф1.	Код на языке Python содержит слишком длинные функции (>40 строк)
Ф2.	Код на языке JavaScript содержит слишком длинные функции (>40 строк)
Ф3.	Код на языке C# содержит слишком длинные функции (>40 строк)
Ф4.	Код на языке Java содержит слишком длинные функции (>40 строк)
Ф5.	Код на языке Go содержит слишком длинные функции (>40 строк)
Ф6.	Код на языке Kotlin содержит слишком длинные функции (>40 строк)
Ф7.	Код на языке Rust содержит слишком длинные функции (>40 строк)
Аргументы функций	
А1.	Код на Python содержит функции с большим числом аргументов (>7)
А2.	Код на JavaScript содержит функции с большим числом аргументов (>7)

№	Описание правила
А3.	Код на C# содержит функции с большим числом аргументов (>7)
А4.	Код на Java содержит функции с большим числом аргументов (>7)
А5.	Код на Go содержит функции с большим числом аргументов (>7)
А6.	Код на Kotlin содержит функции с большим числом аргументов (>7)
А7.	Код на Rust содержит функции с большим числом аргументов (>7)
Комментарии вместо документации	
К1.	Код на Python содержит комментарии вместо документации docstring
К2.	Код на JavaScript содержит комментарии вместо документации JSDoc
К3.	Код на C# содержит комментарии вместо документации XMLDoc
К4.	Код на Java содержит комментарии вместо документации Javadoc
К5.	Код на Go содержит комментарии вместо документации Godoc
К6.	Код на Kotlin содержит комментарии вместо документации KDoc
К7.	Код на Rust содержит комментарии вместо документации rustdoc
Цикломатическая сложность кода	
Ц1.	Цикломатическая сложность кода на языке Python превышена (>10)
Ц2.	Цикломатическая сложность кода на языке JavaScript превышена (>10)
Ц3.	Цикломатическая сложность кода на языке C# превышена (>10)
Ц4.	Цикломатическая сложность кода на языке Java превышена (>10)
Ц5.	Цикломатическая сложность кода на языке Go превышена (>10)
Ц6.	Цикломатическая сложность кода на языке Kotlin превышена (>10)
Ц7.	Цикломатическая сложность кода на языке Rust превышена (>10)
Магические числа	
М1.	Код на Python содержит магические числовые литералы в сравнениях
М2.	Код на JavaScript содержит магические числовые литералы в сравнениях
М3.	Код на C# содержит магические числовые литералы в сравнениях
М4.	Код на Java содержит магические числовые литералы в сравнениях
М5.	Код на Go содержит магические числовые литералы в сравнениях
М6.	Код на Kotlin содержит магические числовые литералы в сравнениях
М7.	Код на Rust содержит магические числовые литералы в сравнениях
Имена идентификаторов	
И1.	Код на Python содержит имена, не соответствующие стандарту PEP8
И2.	Код на JavaScript содержит имена, нарушающие соглашения ESLint
И3.	Код на C# содержит имена, нарушающие соглашения .NET Runtime

№	Описание правила
И4.	Код содержит имена, нарушающие стандартные соглашения Java
И5.	Код содержит имена, нарушающие стандартные соглашения Go
И6.	Код содержит имена, нарушающие стандартные соглашения Kotlin
И7.	Код содержит имена, нарушающие стандартные соглашения Rust

Вариант №1

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `username@hostname:~$`.
3. Реализовать парсер, который поддерживает раскрытие переменных окружения реальной ОС (например, `$HOME`).
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: выполняет команды последовательно, ошибочные строки пропускает. При выполнении скрипта

на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является директория на диске пользователя.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Необходимо реализовать команду `vfs-save путь` для сохранения состояния VFS на диск в исходном формате.
5. Если путь к VFS при загрузке эмулятора не указан, то создать VFS по умолчанию в памяти.
6. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
7. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.

2. Реализовать новые команды: `du`, `history`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `rmdir`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №2

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен содержать имя VFS.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является директория на диске пользователя.
3. Необходимо выдать информацию о загруженной VFS по служебной команде `vfs-info` (имя VFS и хеш SHA-256 ее данных).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `tail`, `uniq`, `wc`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `chmod`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №3

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `Эмулятор - [username@hostname]`.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При

выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является CSV-файл. Для двоичных данных используется base64 или аналогичный формат. Необходимо разобраться, как представлять вложенные элементы VFS.
3. Необходимо выдать информацию о загруженной VFS по служебной команде `vfs-info` (имя VFS и хеш SHA-256 ее данных).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `cat`, `tac`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `cp`, `mv`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №4

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `Эмулятор - [username@hostname]`.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: останавливается при первой ошибке. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является JSON-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `who`, `rev`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `rmdir`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №5

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен содержать имя VFS.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является CSV-файл. Для двоичных данных используется base64 или аналогичный формат. Необходимо разобраться, как представлять вложенные элементы VFS.
3. Поддержать вывод сообщения из файла `motd` при старте. Если `motd` существует, то он находится в корне VFS.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `echo`, `tail`, `cat`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `touch`, `mv`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №6

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен содержать имя VFS.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
   - Путь к конфигурационному файлу.
2. Конфигурационный файл в формате TOML содержит:

- Путь к физическому расположению VFS.
- Путь к стартовому скрипту.

3. Приложение считывает параметры из командной строки и конфигурационного файла.
4. Логика приоритетов: значения из командной строки имеют приоритет над значениями из файла.
5. Сообщить об ошибке чтения конфигурационного файла.
6. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
7. Сообщить об ошибке во время исполнения стартового скрипта.
8. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
9. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является JSON-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.

6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `tree`, `tail`, `history`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mkdir`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №7

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `Эмулятор - [username@hostname]`.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: останавливается при первой ошибке. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является XML-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `cat`, `find`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `chown`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №8

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно содержать имя VFS.
3. Реализовать парсер, который поддерживает раскрытие переменных окружения реальной ОС (например, `$HOME`).
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При

выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Реализовать вывод параметров эмулятора в формате ключ-значение с помощью служебной команды `conf-dump`.
5. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является ZIP-архив. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `tac`, `du`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `chown`, `cp`.
2. По команде `help` выдать список команд с описанием их работы.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №9

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен содержать имя VFS.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к лог-файлу.
   - Путь к стартовому скрипту.
2. Реализовать логирование событий вызова команд в файле формата XML. Событие лога дополнительно содержит:
   - Сообщение о возникшей ошибке.

3. Стартовый скрипт для выполнения команд эмулятора: выполняет команды последовательно, ошибочные строки пропускает. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
4. Реализовать вывод параметров эмулятора в формате ключ-значение с помощью служебной команды `conf-dump`.
5. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является директория на диске пользователя.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `head`, `uname`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `chown`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №10

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `username@hostname:~$`.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При

выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является CSV-файл. Для двоичных данных используется base64 или аналогичный формат. Необходимо разобраться, как представлять вложенные элементы VFS.
3. Необходимо выдать информацию о загруженной VFS по служебной команде `vfs-info` (имя VFS и хеш SHA-256 ее данных).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `cat`, `head`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `touch`, `rm`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №11

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `Эмулятор - [username@hostname]`.
3. Реализовать парсер, который поддерживает раскрытие переменных окружения реальной ОС (например, `$HOME`).
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Пользовательское приглашение к вводу, которое будет отображаться в REPL.

– Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является JSON-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Поддержать вывод сообщения из файла `motd` при старте. Если `motd` существует, то он находится в корне VFS.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `tree`, `clear`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `rm`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №12

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `Эмулятор - [username@hostname]`.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к лог-файлу.
   - Путь к стартовому скрипту.
2. Реализовать логирование событий вызова команд в файле формата JSON. Событие лога дополнительно содержит:

- Сообщение о возникшей ошибке.
- Имя пользователя.

3. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
4. Сообщить об ошибке во время исполнения стартового скрипта.
5. Реализовать вывод параметров эмулятора в формате ключ-значение с помощью служебной команды `conf-dump`.
6. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является JSON-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `uptime`, `head`, `tac`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mv`, `cp`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №13

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно содержать имя VFS.
3. Реализовать парсер, который поддерживает раскрытие переменных окружения реальной ОС (например, `$HOME`).
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: выполняет команды последовательно, ошибочные строки пропускает. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Реализовать вывод параметров эмулятора в формате ключ-значение с помощью служебной команды `conf-dump`.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является JSON-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `cal`, `uname`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mkdir`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №14

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен содержать имя VFS.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: выполняет команды последовательно, ошибочные строки пропускает. При выполнении скрипта

на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является CSV-файл. Для двоичных данных используется base64 или аналогичный формат. Необходимо разобраться, как представлять вложенные элементы VFS.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `date`, `tree`, `cal`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `rm`, `chown`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №15

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен содержать имя VFS.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: останавливается при первой ошибке. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является ZIP-архив. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `whoami`, `wc`, `head`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `rm`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №16

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен содержать имя VFS.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Пользовательское приглашение к вводу, которое будет отображаться в REPL.
   - Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: останавливается при первой ошибке. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является XML-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `uptime`, `who`, `echo`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `chown`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №17

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно содержать имя VFS.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: выполняет команды последовательно, ошибочные строки пропускает. При выполнении скрипта

на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является XML-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `cal`, `echo`, `whoami`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `touch`.
2. По команде `help` выдать список команд с описанием их работы.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №18

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно содержать имя VFS.
3. Реализовать парсер, который поддерживает раскрытие переменных окружения реальной ОС (например, `$HOME`).
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является директория на диске пользователя.
3. Поддержать вывод сообщения из файла `motd` при старте. Если `motd` существует, то он находится в корне VFS.
4. Реализовать команду `vfs-init`, которая заменяет текущую VFS на VFS по умолчанию. В этом случае очищается и физическое представление VFS.
5. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
6. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `cat`, `who`, `uniq`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mkdir`, `rm`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №19

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `username@hostname:~$`.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является JSON-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Реализовать команду `vfs-init`, которая заменяет текущую VFS на VFS по умолчанию. В этом случае очищается и физическое представление VFS.
5. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
6. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.

2. Реализовать новые команды: `uptime`, `find`, `echo`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `chmod`.
2. По команде `help` выдать список команд с описанием их работы.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №20

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `Эмулятор - [username@hostname]`.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: останавливается при первой ошибке. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Реализовать вывод параметров эмулятора в формате ключ-значение с помощью служебной команды `conf-dump`.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является XML-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Если путь к VFS при загрузке эмулятора не указан, то создать VFS по умолчанию в памяти.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `clear`, `tac`, `uname`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mkdir`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №21

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `username@hostname:~$`.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При

выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является ZIP-архив. Для двоичных данных используется base64 или аналогичный формат.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `date`, `tail`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `rmdir`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №22

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен содержать имя VFS.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является JSON-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `cat`, `du`, `tail`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mv`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №23

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `Эмулятор - [username@hostname]`.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к лог-файлу.
   - Путь к стартовому скрипту.
2. Реализовать логирование событий вызова команд в файле формата JSON. Событие лога дополнительно содержит:

- Сообщение о возникшей ошибке.
- Дата и время события.

3. Стартовый скрипт для выполнения команд эмулятора: останавливается при первой ошибке. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является директория на диске пользователя.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `echo`, `whoami`, `find`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mv`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №24

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно содержать имя VFS.
3. Реализовать парсер, который поддерживает раскрытие переменных окружения реальной ОС (например, `$HOME`).
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При

выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является JSON-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `tac`, `tail`, `whoami`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `touch`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №25

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `username@hostname:~$`.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к лог-файлу.
   - Путь к стартовому скрипту.
   - Путь к конфигурационному файлу.
2. Конфигурационный файл в формате JSON содержит:

- Путь к физическому расположению VFS.
- Путь к лог-файлу.
- Путь к стартовому скрипту.

3. Приложение считывает параметры из командной строки и конфигурационного файла.
4. Логика приоритетов: значения из файла имеют приоритет над значениями из командной строки.
5. Реализовать логирование событий вызова команд в файле формата XML. Событие лога дополнительно содержит:
   - Имя пользователя.
6. Стартовый скрипт для выполнения команд эмулятора: выполняет команды последовательно, ошибочные строки пропускает. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
7. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является CSV-файл. Для двоичных данных используется base64 или аналогичный формат. Необходимо разобраться, как представлять вложенные элементы VFS.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).

4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `history`, `date`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `touch`, `chown`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №26

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен содержать имя VFS.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: останавливается при первой ошибке. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Сообщить об ошибке во время исполнения стартового скрипта.

4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является ZIP-архив. Для двоичных данных используется base64 или аналогичный формат.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `whoami`, `du`, `tac`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `cp`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №27

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `Эмулятор - [username@hostname]`.
3. Реализовать парсер, который поддерживает раскрытие переменных окружения реальной ОС (например, `$HOME`).
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: выполняет команды последовательно, ошибочные строки пропускает. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является CSV-файл. Для двоичных данных используется base64 или аналогичный формат. Необходимо разобраться, как представлять вложенные элементы VFS.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `rev`, `uniq`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mv`, `rmdir`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №28

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `username@hostname:~$`.
3. Реализовать парсер, который поддерживает раскрытие переменных окружения реальной ОС (например, `$HOME`).
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к лог-файлу.
   - Путь к стартовому скрипту.
2. Реализовать логирование событий вызова команд в файле формата JSON. Событие лога дополнительно содержит:

– Сообщение о возникшей ошибке.

3. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
4. Сообщить об ошибке во время исполнения стартового скрипта.
5. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является XML-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.

2. Реализовать новые команды: `wc`, `clear`, `date`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `rmdir`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №29

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен содержать имя VFS.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: выполняет команды последовательно, ошибочные строки пропускает. При выполнении скрипта

на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является ZIP-архив. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `find`, `pwd`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `rm`.
2. По команде `help` выдать список команд с описанием их работы.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №30

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `username@hostname:~$`.
3. Реализовать парсер, который поддерживает раскрытие переменных окружения реальной ОС (например, `$HOME`).
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является JSON-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `rev`, `wc`, `uname`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mkdir`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №31

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `Эмулятор - [username@hostname]`.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При

выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является директория на диске пользователя.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `whoami`, `wc`, `uniq`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `chmod`, `rm`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №32

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно содержать имя VFS.
3. Реализовать парсер, который поддерживает раскрытие переменных окружения реальной ОС (например, `$HOME`).
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: выполняет команды последовательно, ошибочные строки пропускает. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является директория на диске пользователя.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `uname`, `tac`, `tree`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `touch`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №33

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `Эмулятор - [username@hostname]`.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При

выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Реализовать вывод параметров эмулятора в формате ключ-значение с помощью служебной команды `conf-dump`.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является директория на диске пользователя.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `wc`, `pwd`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `chown`, `rm`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №34

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `Эмулятор - [username@hostname]`.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: останавливается при первой ошибке. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является JSON-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `echo`, `head`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `cp`, `touch`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №35

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно содержать имя VFS.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: выполняет команды последовательно, ошибочные строки пропускает. При выполнении скрипта

на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является JSON-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `du`, `tail`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mv`, `touch`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №36

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно содержать имя VFS.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При

выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является директория на диске пользователя.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Поддержать вывод сообщения из файла `motd` при старте. Если `motd` существует, то он находится в корне VFS.
5. Реализовать команду `vfs-init`, которая заменяет текущую VFS на VFS по умолчанию. В этом случае очищается и физическое представление VFS.
6. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
7. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.

2. Реализовать новые команды: `wc`, `cal`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `rm`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №37

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `username@hostname:~$`.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: останавливается при первой ошибке. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является XML-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Необходимо выдать информацию о загруженной VFS по служебной команде `vfs-info` (имя VFS и хеш SHA-256 ее данных).
4. Необходимо реализовать команду `vfs-save путь` для сохранения состояния VFS на диск в исходном формате.
5. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
6. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.

2. Реализовать новые команды: `who`, `tail`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mkdir`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №38

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `username@hostname:~$`.
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: останавливается при первой ошибке. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является XML-файл. Для двоичных данных используется base64 или аналогичный формат.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
4. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `wc`, `cal`.

3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `chown`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №39

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен содержать имя VFS.
3. Реализовать парсер, который поддерживает раскрытие переменных окружения реальной ОС (например, `$HOME`).
4. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
5. Реализовать команду `exit`.
6. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.
2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.

3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является ZIP-архив. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
5. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.
2. Реализовать новые команды: `cal`, `head`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mkdir`, `rmdir`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №40

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме консольного интерфейса (CLI).
2. Приглашение к вводу должно формироваться на основе реальных данных ОС, в которой исполняется эмулятор. Пример: `username@hostname:~$`.
3. Реализовать парсер, который корректно обрабатывает аргументы в кавычках.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддержать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
   - Путь к физическому расположению VFS.
   - Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: выполняет команды последовательно, ошибочные строки пропускает. При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является ZIP-архив. Для двоичных данных используется base64 или аналогичный формат.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Поддержать вывод сообщения из файла `motd` при старте. Если `motd` существует, то он находится в корне VFS.
5. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
6. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.

2. Реализовать новые команды: `rev`, `cat`, `du`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `mv`, `rm`.
2. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.